

LARGE LANGUAGE MODELS

A few notes...

- LLMs are an ever-evolving field
 - Information can be outdated monthly, but our goal is to take a look at many of the components that make up current LLMs
 - Terms can be used different ways in different research, so be aware
 - SoTA changes consistently
 - I try to stay up-to-date, but things change quickly, so there is a chance I might mispeak. I have included multiple links to blogs and papers for further reading.
- Read research coming out to be most up-to-date

Review – Language Models

- A language model estimates the probability of a token or sequence of tokens occurring within a longer sequence of tokens.
- N-Gram Language Models
 - o Limitations?
- RNNs
 - o Limitations?
- Transformers

Large Language Models

- Large language models (LLMs) predict a token or sequence of tokens, sometimes many paragraphs worth of predicted tokens.
(Autoregression)
 - LLMs contain far more parameters than recurrent models.
 - LLMs gather far more context.
- Use Transformer Architecture
 - Mainly **Decoder**

Common Large Language Models

Model	Typical reported size(s)	Certainty / source
GPT-4 (OpenAI)	~1.7–1.8 trillion (community estimates)	<i>Estimate</i> — OpenAI has not publicly confirmed parameter counts; community estimates cited. (Exploding Topics)
GPT-OSS / o4 / o3 family (OpenAI public variants)	GPT-OSS: ~120B (open-weight release), o4/o3-mini variants much smaller	<i>Official for GPT-OSS release</i> (OpenAI announcement / reporting). (The Verge)
Claude 3 family (Anthropic: Haiku / Sonnet / Opus / 3.7 Sonnet, etc.)	Undisclosed (widely believed to be in the hundreds of billions for larger variants)	Anthropic publishes model family names but does not publish exact parameter counts ; community estimates exist. (Anthropic)
Gemini 1.5 / Gemini family (Google / DeepMind)	Undisclosed (Google does not publish exact parameter counts for Gemini tiers)	Google describes different “Pro/Ultra/Flash” variants and capabilities; parameter counts not publicly disclosed. (blog.google)
LLaMA 3 / LLaMA 3.1 (Meta)	8B, 70B, 405B (3.1 release includes 8B / 70B / 405B)	Official Meta releases / Bedrock & Hugging Face reporting. (AI Meta)
LLaMA 2 (Meta)	7B, 13B, 70B	Official (Meta / Hugging Face model cards). (Hugging Face)
Mistral 7B (Mistral AI)	~7.3B	Official release by Mistral. (Mistral AI)
Mistral Large 2 (Mistral-Large-2407)	123B	Official Mistral announcement / partner coverage. (Mistral AI)
PaLM (Google)	~540B (original PaLM)	Public documentation / academic reporting for original PaLM. (PaLM2 reporting varies by source.) (Wikipedia)
OPT (Meta)	175B (OPT-175B)	Official Meta release of OPT-175B. (AI Meta)
BLOOM (BigScience)	176B	Official paper / release. (arXiv)
Falcon-40B (TII)	40B	Official (TII / Hugging Face model card). (Falcon LLM)
GPT-3 (OpenAI)	175B	Official (GPT-3 paper). Useful historical baseline. (arXiv)

Model	Training Data Summary	Notes
GPT-4 / GPT-4o / GPT-5 (OpenAI)	Not publicly disclosed; widely believed to include: • Common Crawl (filtered web text) • Books (licensed & public domain) • Academic papers (e.g. arXiv, PubMed) • Code repositories (GitHub, etc.) • Synthetic data from GPT-4 self-training	OpenAI also uses reinforcement learning from human feedback (RLHF) and model self-distillation (generated data for reasoning & alignment).
Claude 3 (Anthropic)	Mix of filtered web data, books, Wikipedia, code, and Constitutional AI–filtered dialogues	Anthropic emphasizes “harmlessness, honesty, helpfulness” via post-training on curated text and synthetic dialogue.
Gemini 1.5 (Google DeepMind)	Built on Google’s GLaM + PaLM + Gemini data pipeline: • Web documents, YouTube transcripts, news • BooksCorpus, Wikipedia • Code (GitHub, internal datasets) • Multimodal data (images, video, audio transcriptions)	Uses massive multimodal pretraining , unique among the group.
LLaMA 3 (Meta)	Disclosed aggregate stats: ~15T tokens • Refined Common Crawl (“RedPajama-style”) • GitHub code • Wikipedia, books • StackExchange, arXiv	Meta emphasized aggressive deduplication and filtering for high-quality text.
Mistral / Mixtral	Uses a curated mix of open web + code + math + dialogue . Exact composition undisclosed, but inferred similar to RedPajama and The Pile.	Mixtral uses a sparse MoE decoder , so data diversity helps expert specialization.
(*table created with help of generate AI)		

COMMON LLM TRAINING DATA

Benefits of Large Language Models

- Can generate clear, easy-to-understand text for a wide variety of target audiences.
- Can make predictions on tasks they are explicitly trained on.
- Claims that LLMs can also make predictions for input they were not explicitly trained on.

Benchmarking LLMs

- Standardized frameworks for assessing LLMs
- Leaderboards often found online for current SoTA

[MMLU](#), [LiveBench](#), [More](#)

Model	Organization	Global Average	Reasoning Average	Coding Average	Agentic Coding Average	Mathematics Average	Data Analysis Average
GPT-5 High	OpenAI	79.33	98.17	77.10	46.67	92.77	71.63
GPT-5 Medium	OpenAI	78.85	96.58	75.05	50.00	89.95	72.38
GPT-5 Pro	OpenAI	78.73	96.67	72.11	43.33	93.77	72.42
Claude Sonnet 4.5 Thinking	Anthropic	78.26	95.28	80.36	50.00	92.96	71.76
GPT-5 Codex	OpenAI	78.24	98.67	69.61	48.33	92.74	70.29
GPT-5 Mini High	OpenAI	75.31	91.44	68.20	43.33	90.69	71.95
Claude 4.1 Opus Thinking	Anthropic	75.25	93.19	74.66	45.00	91.16	71.14
GPT-5 Low	OpenAI	74.65	90.47	74.28	35.00	85.33	69.72
Claude 4 Sonnet Thinking	Anthropic	73.82	95.25	77.48	38.33	85.25	69.84
Grok 4	xAI	72.84	97.78	73.13	26.67	88.84	69.53
Gemini 2.5 Pro (Max Thinking)	Google	71.92	94.28	75.69	25.00	84.19	71.50

Fine-tuning Large Language Models

- Pattern-recognition abilities of foundation language models require relatively little additional training to learn specific tasks.
- That additional training helps the model make better predictions on a specific task. **Fine-tuning.**
- Still can be computationally expensive.
 - o Involves updating the weight and bias of every parameter on each backpropagation iteration.

MORE EFFICIENTLY FINE- TUNING LARGE LANGUAGE MODELS

Distillation

Distillation creates a smaller version of an LLM. (DeepSeek-R1-Distill, Distilbert)

- Faster, requires less resources, can be slightly worse at predictions
- Most common form of distillation uses bulk inference to label data.
- Labeled data is then used to train a new, smaller model (known as the student model) that can be more affordably served.
- The labeled data serves as a channel by which the larger model (known as the teacher model) funnels its knowledge to the smaller model.

PEFT

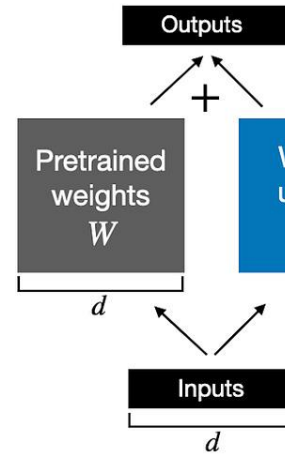
Parameter-efficient tuning (PEFT) can fine-tune an LLM by adjusting only a subset of parameters on each backpropagation iteration.

Example – LoRA

LoRA -Low-Rank Adaptation

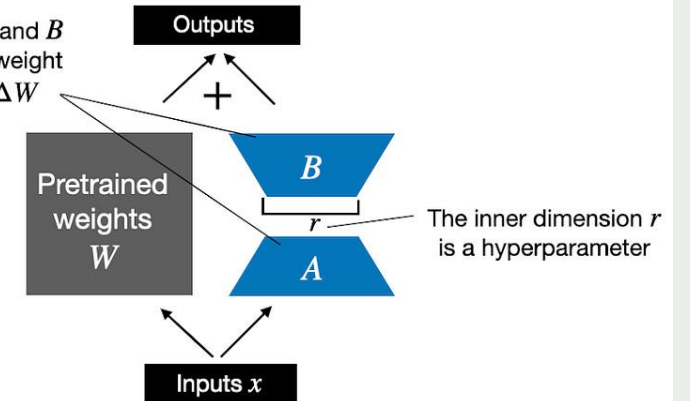
- Decompose weight changes into a lower-rank representation.
- If A has the same number of rows as ΔW and B has the same number of columns as ΔW , we can write the decomposition as $\Delta W = AB$. (AB is the matrix multiplication result between matrices A and B .)
- ([Paper](#))

Weight update in **regular finetuning**



LoRA matrices A and B approximate the weight update matrix ΔW

Weight update in **LoRA**



	TruthfulQA MC1	TruthfulQA MC2	BLiMP Causative	MMLU Global Facts
Llama 2 7B base	0.253	0.397	0.787	0.320
LoRA default 1	0.288	0.421	0.750	0.270
LoRA default 2	0.284	0.422	0.747	0.260
LoRA default 3	0.282	0.421	0.747	0.270

QLoRA

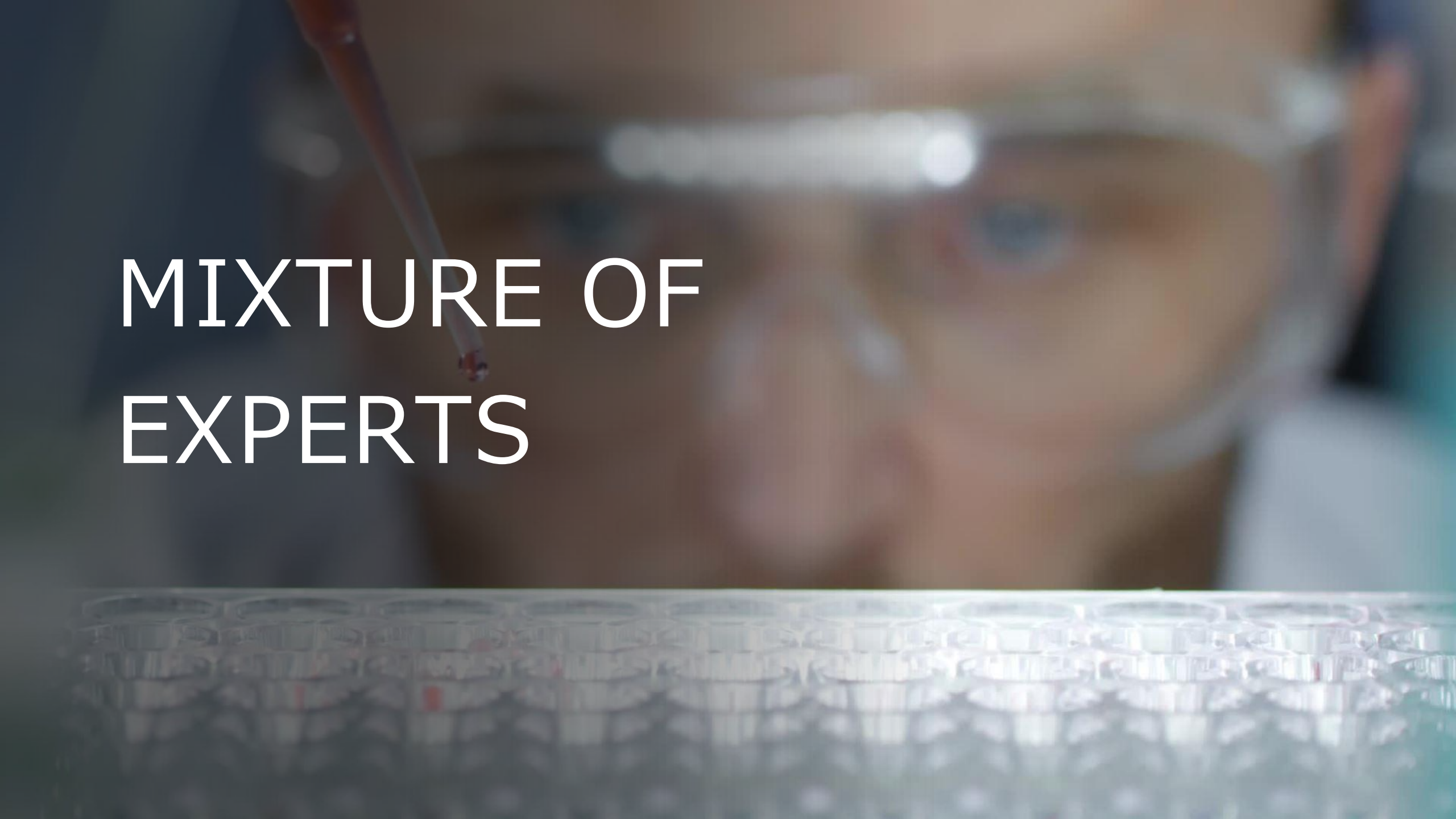
- QLoRA quantizes the pretrained weights to 4-bit precision and uses paged optimizers to handle memory spikes.

([Paper](#))

	TruthfulQA MC1	TruthfulQA MC2	BLiMP Causative	MMLU Global Facts
Llama 2 7B base	0.253	0.397	0.787	0.320
LoRA default	0.288	0.421	0.750	0.270
QLoRA (nf4)	0.280	0.414	0.783	0.230

More LoRA

<https://aclanthology.org/>



MIXTURE OF EXPERTS

Mixture of Experts

Aim to pretrain models with less compute

- Allows to increase data or model size with same compute budget

Rather than using dense feed-forward layers, use sparse MoE layers.

- Each MoE layer has a number of "experts", where each expert is a neural network.
- Also include a gate network or router which determines which tokens are sent to which expert. ([More Reading](#))

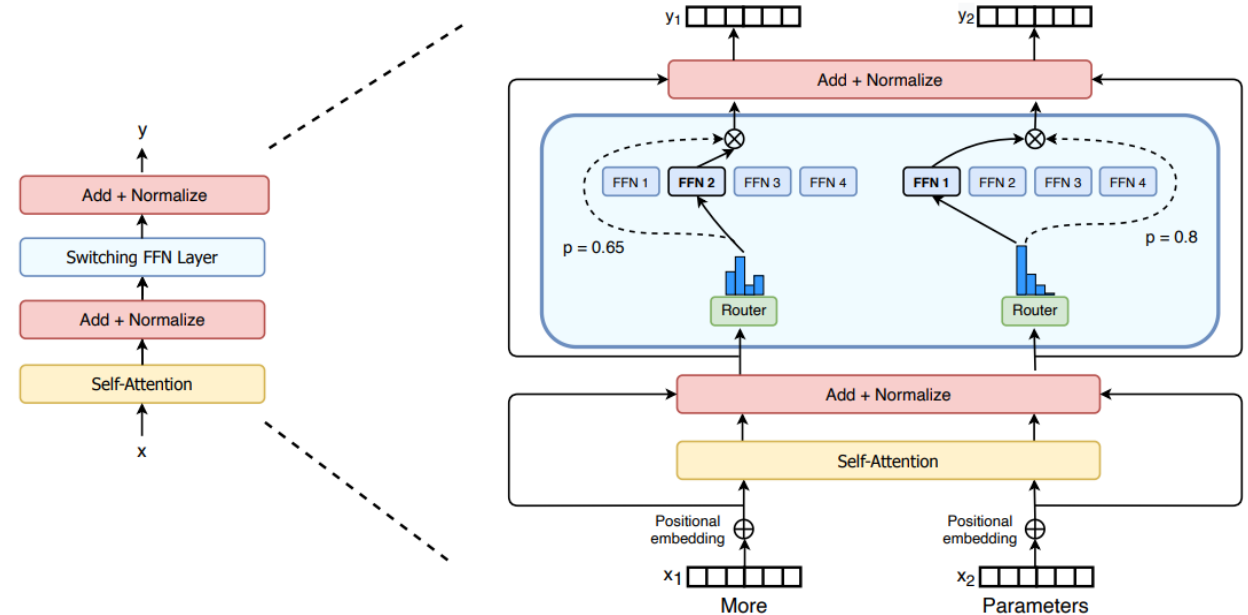


Figure 2: Illustration of a Switch Transformer encoder block. We replace the dense feed forward network (FFN) layer present in the Transformer with a sparse Switch FFN layer (light blue). The layer operates independently on the tokens in the sequence. We diagram two tokens (x_1 = "More" and x_2 = "Parameters" below) being routed (solid lines) across four FFN experts, where the router independently routes each token. The switch FFN layer returns the output of the selected FFN multiplied by the router gate value (dotted-line).

Offline Inference

Online inference – the LLM generates predictions on demand. (Waits for the user to prompt).

- LLMs are large, so can take a while to return a prediction.

Offline inference – predict ahead of time which queries will be made and store them for later.

- E.g. Questions about a presidential candidate
- Information/queries during flu season

PROMPT ENGINEERING

Prompt Engineering

Prompt engineering enables an LLM's end users to customize the model's output.

- Humans learn well from examples. So do LLMs. Showing one example to an LLM is called **one-shot prompting**.

Prompt Engineering – One-shot

Suppose you want a model to use the following format to output a fruit's family:

```
peach: drupe  
apple: pome -
```

Prompt Engineering – Few-shot

Sometimes a single example is insufficient. That is, the user must show the LLM multiple examples. This is called **few-shot** prompting.

```
plum: drupe  
pear: pome  
lemon: ----
```

Prompt Engineering – Zero-shot

In theory, LLMs can perform tasks not explicitly fine-tuned on or given examples for. This is called **zero-shot** prompting.

"Output the following fruit's family in the form fruit:family.

Apple"

LLM IMPROVEMENTS

How can LLMs
improve beyond
more training data
and size?

Retrieval Augmented Generation (RAG)

LLMs are limited in knowledge to what they have been pre-trained or fine-tuned on.

- If a model is trained up to August 2024. It cannot (alone) answer questions about current events.
- If prompted, it might hallucinate an answer.

What is the solution?

- Retraining an LLM every month would be computationally expensive.
- **RAG**

Retrieval Augmented Generation (RAG)

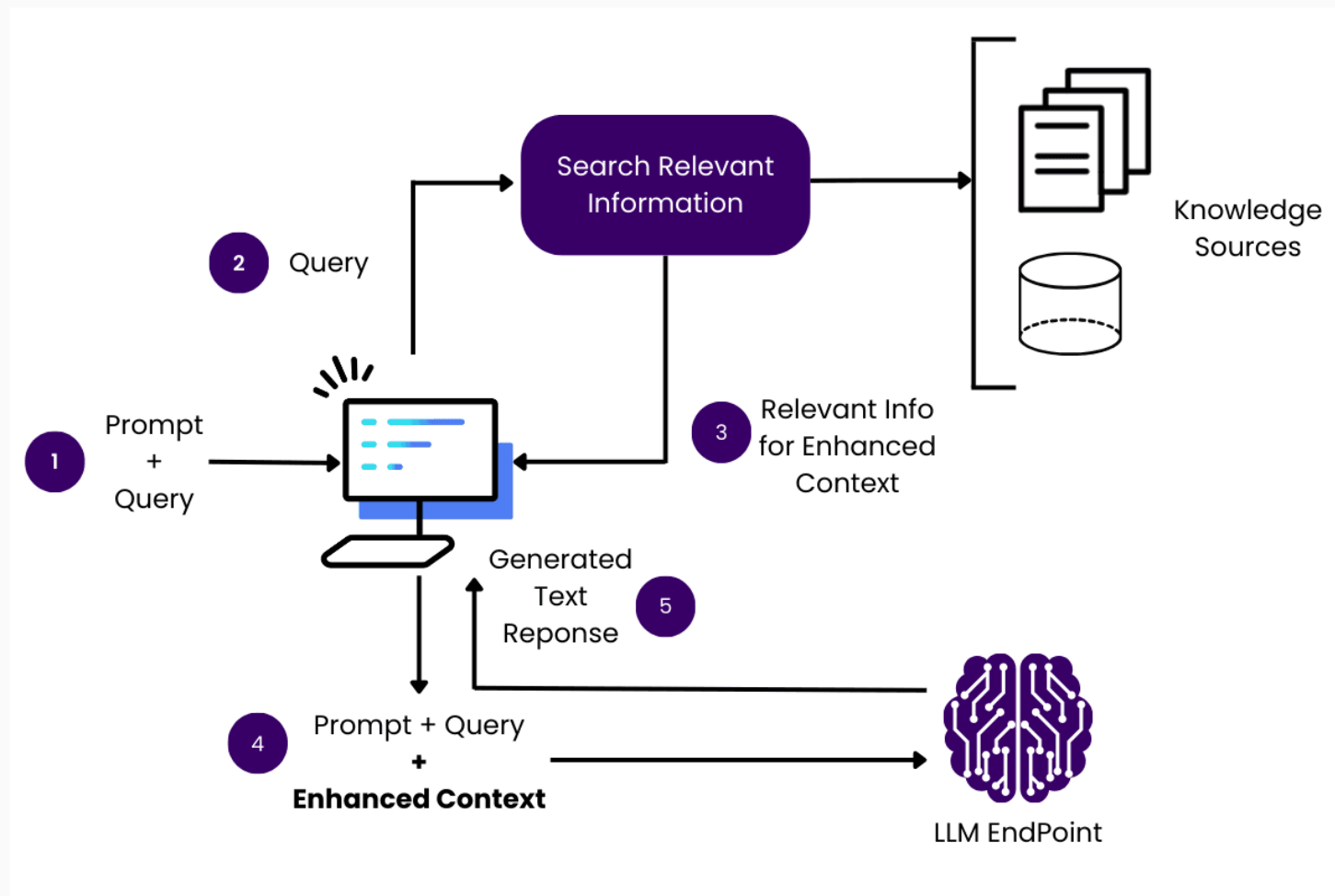
- **RAG** (Retrieval-Augmented Generation) is an AI framework that combines the strengths of traditional information retrieval systems (such as search and databases) with the capabilities of generative large language models (LLMs).
- By combining your data and world knowledge with LLM language skills, grounded generation is more accurate, up-to-date, and relevant to your specific needs.

(Sources: [Google Cloud](#), [AWS](#))

RAG – How is new data added?

- Any new data can be added to enhance an LLM's knowledge/response.
- For external data (web pages, documents, etc. outside the original training data), pre-process and convert it into vectors via embedding language models ([Qwen Embedding Model](#)) and store in a **knowledge base**.
- For each user prompt, convert the prompt into a query vector and find similar vectors in knowledge base.
- Use most similar documents as additional input alongside prompt.

Retrieval Augmented Generation (RAG)



RAG – Benefits

- Access to fresh information.
 - Up-to-date information (similar to you looking up additional information to answer a question)
- Can ground responses with citations

Web Search

Leverage APIs (e.g. google) to search the web for more relevant information during processing.

- Allows for greater flexibility than RAG
- More prone to misinformation and hallucinations without secondary checks

REASONING

Reasoning

If LLMs are trained on text generation (auto-regressive), how can they learn to "reason"?

Reasoning Models

- Fine-tune LLMs to break complex problems into smaller steps, prior to generating a final output.
- Rather than immediately generating a direct response to a user's input, reasoning models are trained to first generate intermediate "reasoning steps" before arriving at the final answer provided to the user.
- Some reasoning LLMs show users their reasoning traces, while others only summarize or altogether hide these intermediate outputs.

Sources: [IBM](#), [Secondary](#)

Reasoning

If a train is moving at 60 mph and travels for 3 hours, how far does it go?

The train travels 180 miles.

Plain response

To determine the distance traveled, use the formula:

Distance = Speed $\times$ Time

Given that the speed is 60 mph and the time is 3 hours:

Distance = 60 mph $\times$ 3 hours = 180 miles

So, the train travels 180 miles.

Response with intermediate reasoning steps

Inference- Time: Chain-of- thought

- Use Prompt Engineering to encourage the model to "think through" a problem.
 - "think step by step"
- Encourages model to generate intermediate steps
- ([Paper](#))

Regular prompting

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: The answer (arabic numerals) is

(Output) 8 ✗

Chain-of-thought prompting

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

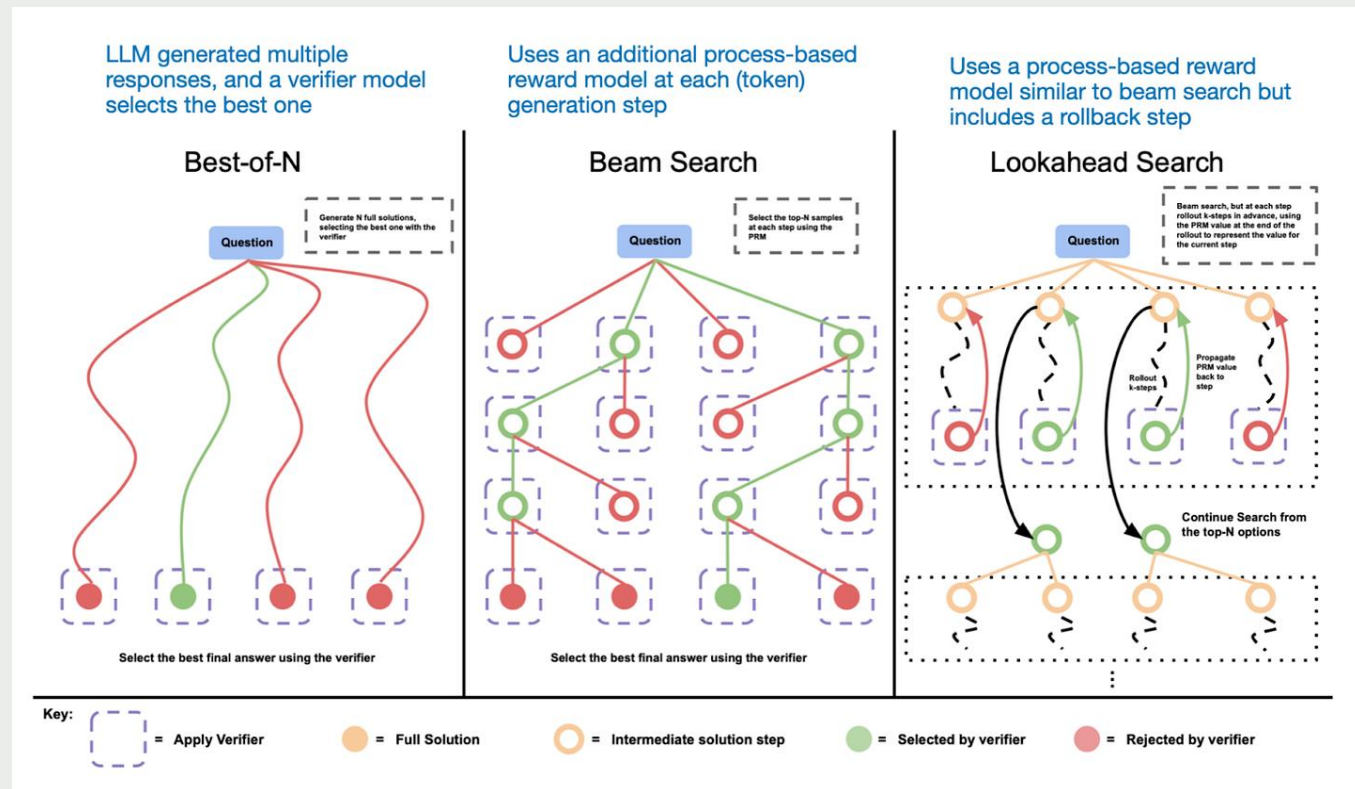
A: **Let's think step by step.**

(Output) *There are 16 balls in total. Half of the balls are golf balls. That means that there are 8 golf balls. Half of the golf balls are blue. That means that there are 4 blue golf balls. ✓*

Inference- Time: Majority Voting

- Have LLM generate multiple responses, vote on response or use AI verifier.

([Paper](#))

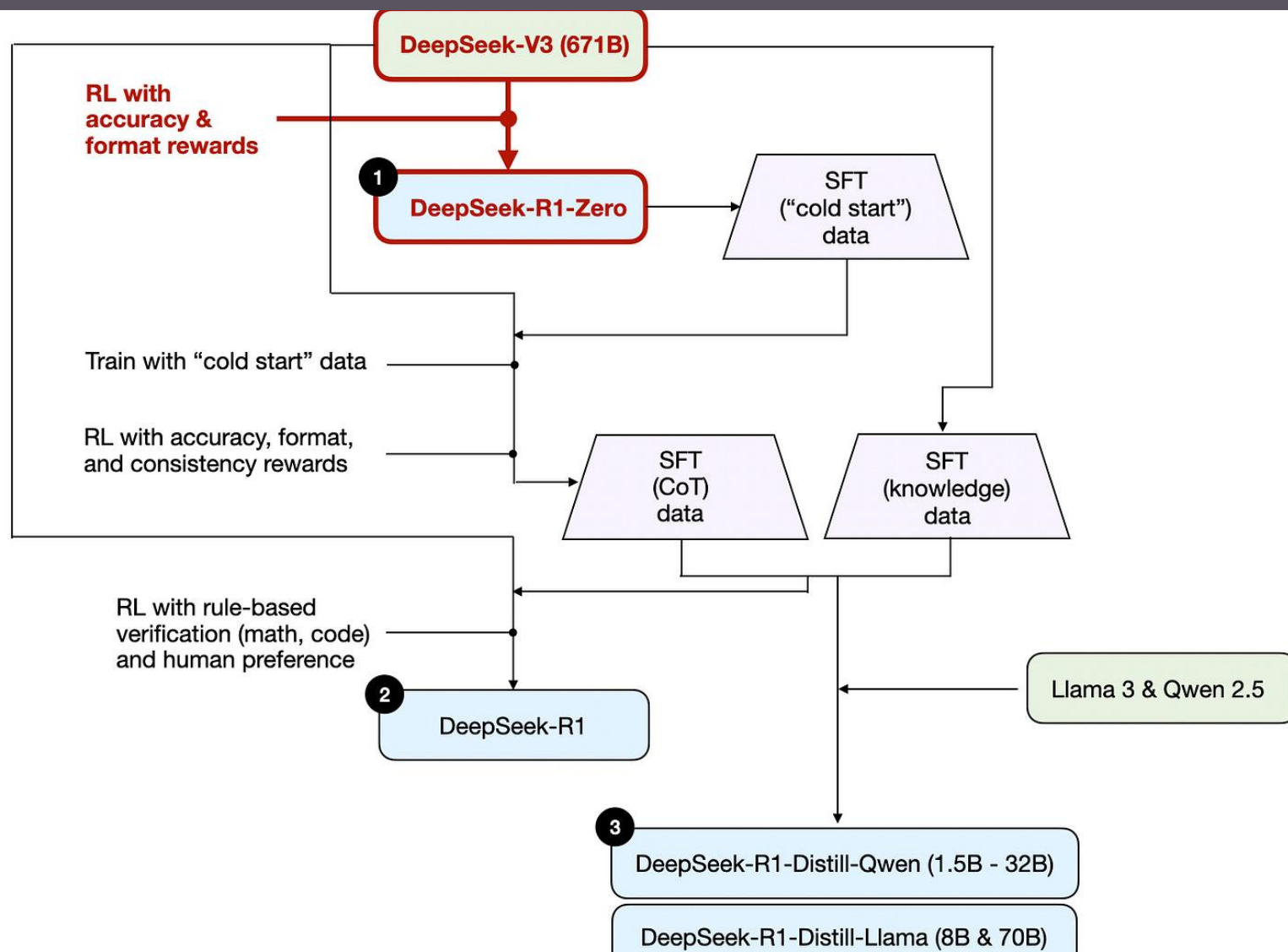


Reinforcement Learning

- Use a reward model to check the LLM response.

Examples:

- **RLHF** (talked about in alignment)
- **DeepSeek**
 - The **accuracy reward** uses the LeetCode compiler to verify coding answers and a deterministic system to evaluate mathematical responses.
 - The **format reward** relies on an LLM judge to ensure responses follow the expected format, such as placing reasoning steps inside <think> tags.



Problems of Large Language Models

Training an LLM entails many problems, including:

- Gathering an enormous training set.
- Consuming multiple months and enormous computational resources and electricity.
- Solving parallelism challenges.

Using LLMs to infer predictions causes the following problems:

- LLMs **hallucinate**, meaning their predictions can contain mistakes.
- LLMs consume enormous amounts of computational resources and electricity.

Training LLMs on larger datasets typically reduces the amount of resources required for inference, though the larger training sets incur more training resources.

- Like all ML models, LLMs can exhibit all sorts of **bias**.

Don't Forget the Bias

Like any form of machine learning, LLMs generally share the biases of:

- The data they were trained on.
- The data they were distilled on.
- Designer Biases

Human/LLM Alignment

- The process of ensuring that large language models (LLMs) behave in ways that are consistent with human values, goals, and expectations.
- Why is this important?

Prevents/Reduces:

- Unwanted/intended behaviors
- Bias
- Unsafe outputs
- Misinformation

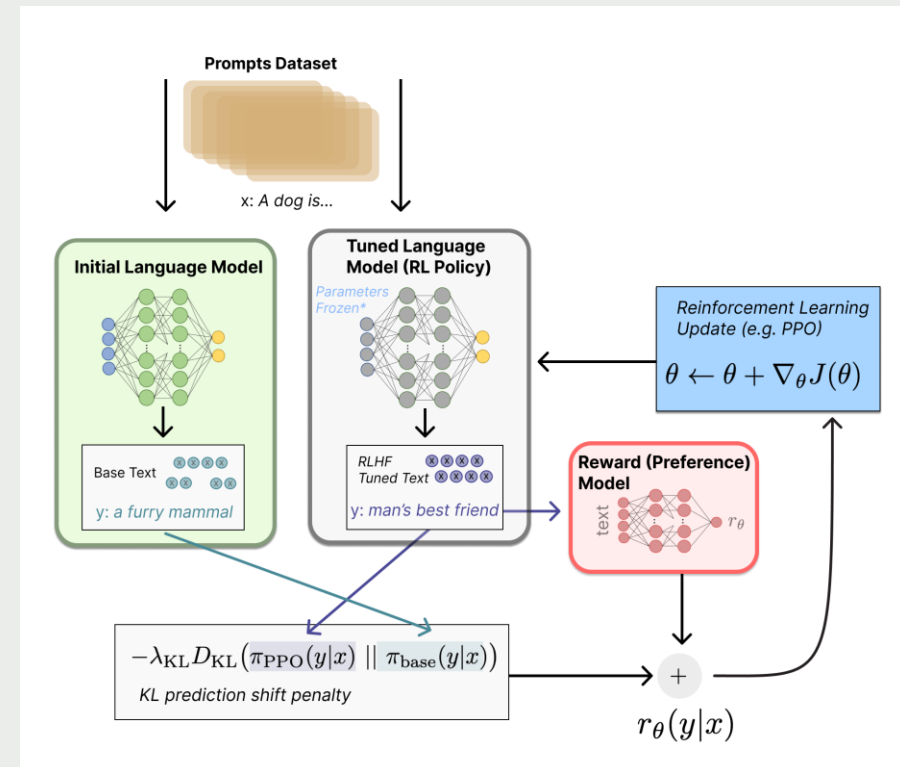
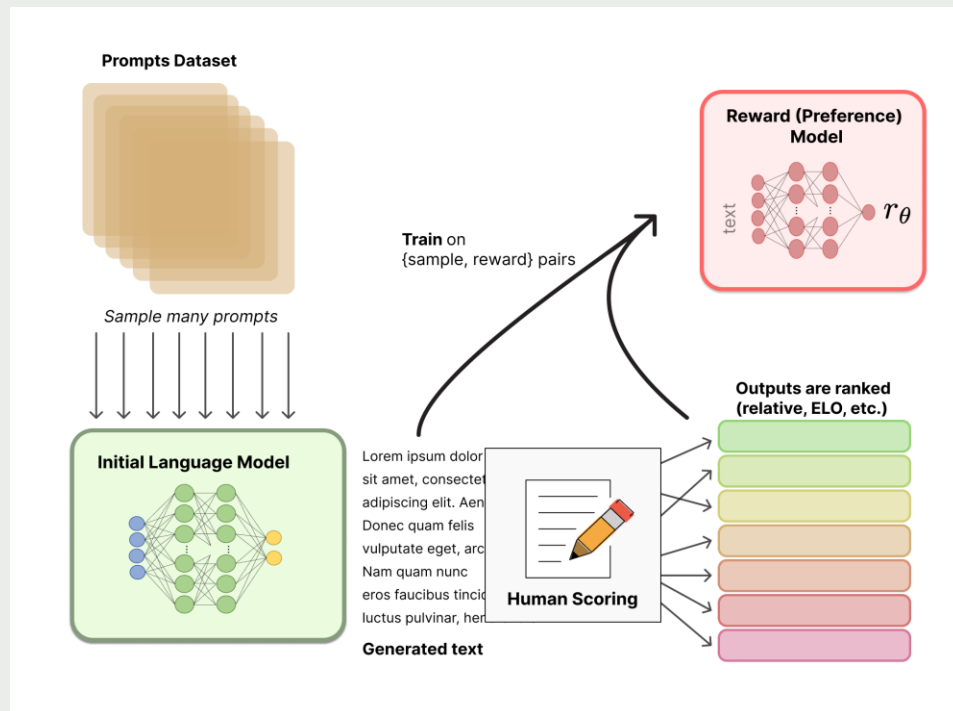
Reinforcement Learning From Human Feedback (RLHF)

Use human feedback to guide an LLM. ([more reading](#))

Human annotators rank LLM outputs

Use rankings to train a reward model

Use reward model to optimize the LLM via reinforcement learning.



source

Example

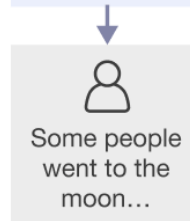
The role of RLHF in ChatGPT

Step 1:
Collect demonstration data and
train a supervised policy.

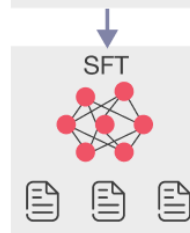
A prompt is
sampled from
our prompt
dataset.



A labeler
demonstrates
the desired
output behavior.



This data is
used to fine-
tune GPT-3 with
supervised
learning.

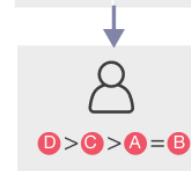


Step 2: Collect comparison
data, and train a reward
model.

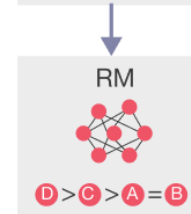
A prompt and
several model
outputs are
sampled.



A labeler
ranks the
output from
best to worst.



This data is
used to train
our reward
model.



Step 3: Optimize a policy
against the reward model
using reinforcement learning.

A new prompt
is sampled
from the
dataset.



The policy
generates an
output.



The reward
model
calculates the
reward for an
output



The reward is
used to update
the policy
using PPO.



Supervised Fine-Tuning

- **Fine-tune the LLM on a dataset of human-annotated examples**
 - Similar idea to previous tasks
- **Focus on specific tasks**
 - Bias
 - Misinformation
 - Privacy protection
 - Etc.

Fine-tuning Datasets

<https://github.com/glgh/awesome-llm-human-preference-datasets>

<https://github.com/mlabonne/llm-datasets>

OTHER LLM MODELS

Instruction Models

Fine-tune models on following instructions

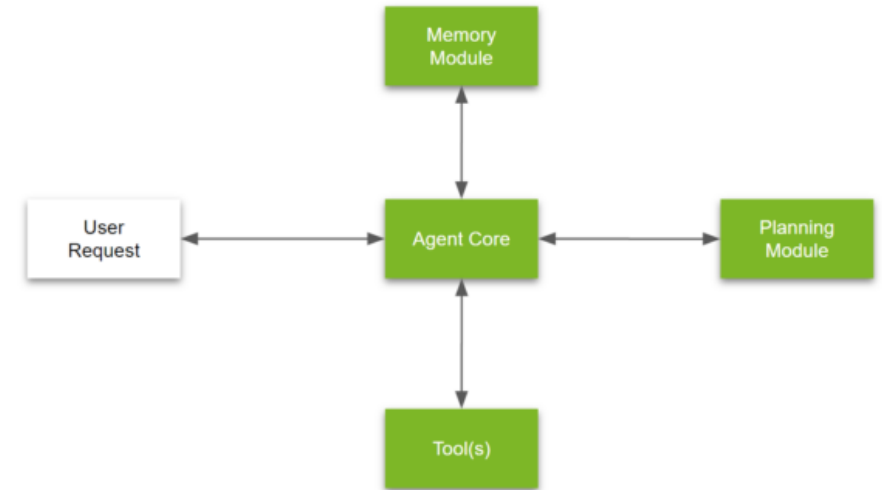
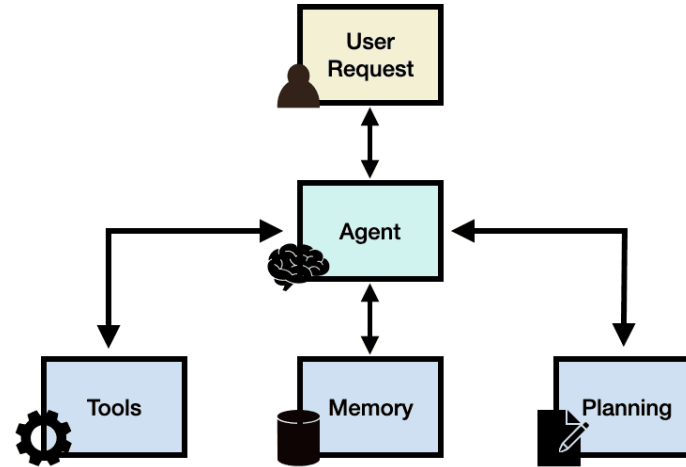
- E.g. Alpaca dataset (<https://huggingface.co/datasets/tatsu-lab/alpaca>)
- Can help LLMs on specific tasks, as well as following instructions in general
- (**More reading**)

```
{  
  "instruction": "Create a classification task by clustering the given list of items.",  
  "input": "Apples, oranges, bananas, strawberries, pineapples",  
  "output": "Class 1: Apples, Oranges\nClass 2: Bananas, Strawberries\nClass 3: Pineapples",  
  "text": "Below is an instruction that describes a task, paired with an input that provides further context."  
}
```

LLM Agents

- Combine LLMs with different modules acting as the controller to decide how to execute specific tasks.
- User Request - a user question or request
- Agent/Brain - the agent core acting as coordinator
- Planning - assists the agent in planning future actions
- Memory - manages the agent's past behaviors

(Additional Reading: [Nvidia](#), [ai-blog](#))



LLMs

Want the most up-to-date
information? Read research papers
for new developments